1. We should use the chat name “Python Review maps_signal_250220”.
2. Now, I would like to review the maps_signal.py for improvements, shortcomings, corrections etc. Before starting, we need to understand a few things which will help us to bring the correct solution. I request you to have discussions on finding improvements before starting the editing of the files. This will reduce the iterations and useless things in chat codes. In case my explanation is not clear, you can also demand more clarity to take the right call.
3. I am assuming that we will follow the above things to avoid iterations of replies and answers.
4. Now I am explaining important variables, they have behaviour control of incoming data from remote, some are used now and some of them will be used in due course of our progress in the development of the application.

a. FragHdrEnable – If true, the fragment will have a header and the fragment size will change accordingly.

b. FRAG_HDR_SIZE – No of words (int16) in receiving serial port data of each fragment, at the start of every Fragment, received from remote. It is currently fixed 6 but may vary in future based on need.

c. FRAG_DATA_SIZE– No of words (int16) in one fragment. It is fixed to 1024. It may change, but always the power of 2. It will be optimized later to optimize the USB CDC communication speed.

d. FRAGMENT_SIZE – The number of words per fragment of each channel. It includes the header and data size.

e. ADC_LSB_UV – it is the value in microvolt for each sample of the ADC, used in time plot, when the y-axis is marked in other than count, i.e., “CNT”.

f. sampleRate – default ADC sampling rate, but will be get updated according to remote after receiving start-up message.

g. no_of_fragments – It is a very important variable. To understand its uses, which will lead to changes in the current code also. Let’s take an example the sampling rate is 2048, and FRAGMENT_SIZE is 1024, considering that the header size is zero. Now, the remote will send the two fragments, each of 1024 words (i.e. 2048 bytes) to complete the 1-second data of the channel. The multiple-channel handling will be taken later once this implementation is finished. ***Point to remember – After the fragment is received, we need to put in a time plot, once all the fragments are received (i.e. one-second data), need to process the FFT and other things***.

h. TIME_WIN_SIZE – no of seconds, data to keep in the plot window, currently it is 5 but it may change based on customer and other needs.

i. samples_in_window – Numbers of samples in the time plot window, i.e., sampleRate * TIME_WIN_SIZE

j. XAXIS_SEC_DIV – number of divisions of 1 second in time plot, currently it is not used, but may be used for minor tick marking in time plot. It is assigned with 5.

k. sys_notch – it is used to request whether remote to switch on the notch filter or not.

l. timeYSpanVal – used to y-axis fix range in time plot, if used, current we are working on dynamically.

m. freqXSpanHz – The Geophone signal range will be less than 500 Hz, even if it is sampled at the higher range, so utilizing the same width of time plot, zoom the 0- freqXSpanHz, it is defined

for FFT plot visualization. So when we calculate the FFT, we get frequency components to half of the sampling rate, which is higher. While displaying the same on the FFT plot, use the above range only, and ignore the rest of the parts. It will fast to display and insert. If FFT allows that required range calculation instead of whole, it will be good but I am aware of this possibility.

n. axisYMajorDiv – For FFT plot, number of major tick divisions.

o. session_status – used to know if the data capturing session is on or off

p. screen_status – it is used for time and frequency plot to freeze (last five seconds) or live streaming. It helps to visualize the data if needed. The rest of the activities will be as it, like capturing and inserting in buffer or some others as well, it controls the screen update only

q. record_status – it controls the incoming data to be stored in a file or not.

r. event_detection – It controls the event detection in every fragment based on event_threshold (in counts to reduce the calculations), if it is true and any sample crosses the event_threshold in either sign, the system considers the event in data and pre, current and post fragment need to store in the event file.

s. event_threshold – it is in the ADC counts for event detection, if detection is enabled.

5. That's all the explanation of all major variables used so far, some are not used now, but will be used or later will deleted.
6. First, we will correct the frequency plot and time plots with the consideration that multiple fragments will be received for the channel. To start, if we keep the header off, the assumption will be that fragments are in order, i.e. 0, 1, …. Later we will add the header when the fragment number and channel number come with each fragment to sync the data for multiple channels from the remote system.
7. Names of files for continuous recording and event recording will be based on the time stamp in the name so, it will be very easy to know the file details by name. For continuous recording, to keep the file size small, it will be based on every hour boundary, it closes and opens a new one. For events, it will be seconds based, if there are two events in one second, add in the last letter as A, B... so on.
8. The nature of data in the file will be integers and text format to readability by open-source tools.
9. Files and saving actions will be taken once all the things are perfect in plot and handling to avoid too much changes in one go. It also helps in debugging the problems.
10. We will start with latest files. If it is possible, keep each full file in one canvas, so that testing for me will be easy, otherwise I have to cut and past in my current files and verify for mistake also. It also limits the number of available canvases in each chat and good thing that every canvas has right, corrected latest code. So, any time, we can come back and make changes if needed in it, no need to have new one.